What are Runnables in LangChain

1. Introduction

As LangChain evolved, developers faced a problem:

- Too many different components (chains, prompts, models, parsers)
- Hard to connect them cleanly

👉 **Runnables** were introduced to **standardize everything**.

---

2. Recap

Before Runnables, you learned:

- Chains (Simple, Sequential, Parallel, Conditional)
- Output Parsers
- Prompts

But:

❌ Each worked differently

❌ Hard to combine and scale

👉 We needed a **unified system**

---

3. Why Do Runnables Exist?

🚨 **Problems Before Runnables:**

- Different interfaces for different components
- Hard to combine chains + models + parsers
- Complex code for multi-step workflows
- Difficult to debug and maintain

✅ **Goal of Runnables:**

Create a **single standard interface** for all components

💡 **Simple Idea:**

Instead of learning many tools separately →

👉 Use one common system: **Runnable**

---

4. Simple LLM App (Before Runnables)

Example Flow:

User Input → Prompt → LLM → Output Parser → Result

❌ **Problem:**

- Each step is separate
- You manually connect everything
- Code becomes messy as app grows

💡 **Analogy:**

Like connecting wires manually every time ❌

---

5. Different Types of Chains

You already know:

- Simple Chain
- Sequential Chain
- Parallel Chain
- Conditional Chain

❌ **Problem:**

Each chain type:

- Has different setup
- Different syntax
- Hard to combine

👉 This leads to confusion in real applications

---

6. Problems (Detailed Understanding)

🚨 **Key Issues:**

1. **No standardization**
    - Each component behaves differently
2. **Poor composability**
    - Hard to combine multiple steps
3. **Scaling difficulty**
    - Large apps become complex
4. **Code duplication**
    - Repeating similar logic

👉 Solution = **Runnables**

---

7. What are Runnables?

✅ **Simple Definition:**

A **Runnable** is:

A standard building block in LangChain that can take input, process it, and return output

---

💡 **Even Simpler:**

👉 Runnable = **Function-like object**

Key Idea:

Everything becomes a Runnable:

- Prompt
- LLM
- Output Parser
- Chain

Basic Flow:

Input → Runnable → Output

Core Concept (VERY IMPORTANT)

✅ **Runnables follow a standard interface:**

They all support:

- .invoke() → run once
- .stream() → stream output
- .batch() → process multiple inputs

Example:

result = runnable.invoke("Hello")

👉 No matter what the component is, you use the **same method**

🔗 **Runnable Composition (Most Powerful Feature)**

✅ **You can connect runnables using | (pipe operator)**

Example:

chain = prompt | model | parser

💡 **Meaning:**

1. Prompt formats input
2. Model generates output
3. Parser structures it

👉 This creates a **clean pipeline**

Visual Flow:

Input → Prompt → Model → Parser → Output

🔄 **Replacing Chains with Runnables**

❌ **Old Way:**

- Use different chain classes

✅ **New Way:**

- Use **Runnable pipelines**

Sequential Chain (Runnable way):

chain = step1 | step2 | step3

Parallel Chain (Runnable way):

```
from langchain_core.runnables import RunnableParallel

chain = RunnableParallel({
  "summary": summary_chain,
  "keywords": keyword_chain
})
```

Conditional Chain:

Use logic to choose runnable dynamically

🚀 **Benefits of Runnables**

✅ **1. Standardization**

- Same interface everywhere

✅ **2. Composability**

- Easily combine components

✅ **3. Simplicity**

- Cleaner, readable code

✅ **4. Scalability**

- Works for small → large apps

✅ **5. Flexibility**

- Supports sequential, parallel, conditional flows

---

8. Code Demo (Conceptual Understanding)

Example Runnable Pipeline:

```
chain = prompt | model | parser

result = chain.invoke("Explain AI")
```

💡 **What happens internally:**

1. Input → Prompt formats it
2. → Model generates response
3. → Parser structures output

✅ **Final Output:**

Clean, structured, ready-to-use data

🔥 **Big Picture Understanding**

Before Runnables:

❌ Many separate tools

❌ Complex connections

After Runnables:

✅ Everything is a Runnable

✅ Easy pipelines using |

✅ Clean and scalable design

🔄 **Real-World Example**

Task: Resume Analyzer

Input:

"This is my resume..."

Runnable Pipeline:

```
chain = prompt | model | parser
```

Output:

```
{
 "name": "Ali",
 "skills": ["Python", "SQL"],
 "experience": 2
}
```

👉 Ready to:

- Store in DB
- Send to API
- Display in UI

⚠️ **Important Points to Remember**

1. Runnable = **standard building block**
2. Everything in LangChain can be a runnable
3. Use .invoke() to run
4. Use | to combine steps
5. Replaces traditional chains
6. Makes code cleaner and scalable

✅ **Final Summary**

- **Runnables unify all LangChain components**
- They make it easy to:
  - Build pipelines
  - Combine steps
  - Scale applications
- Key features:
  - Standard interface (invoke, batch, stream)
  - Pipe operator (|)
- They are the **modern way to build LangChain apps**
